Digital System Design with Verilog

ใบปฏิบัติการที่ 8 (สัปดาห์ที่ 9)

Testbench Design & Functional Verification

— Self-Checking Testbench สำหรับ Counter และ Traffic Light FSM

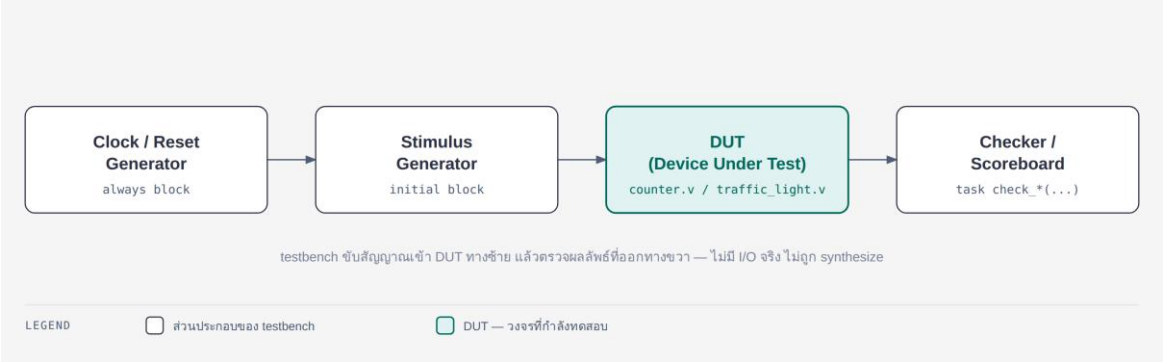
ชื่อ-นามสกุล	รหัสนักศึกษา	กลุ่มปฏิบัติการ	วันที่ส่ง

1. จุดประสงค์การเรียนรู้

- อธิบายโครงสร้างของ testbench (clock/reset generator, stimulus generator, DUT, checker) และเขียน skeleton ได้ด้วยตนเอง
- เขียน self-checking testbench ที่ตัดสิน PASS/FAIL อัตโนมัติด้วย task checker และตัวแปรนับ error
- ใช้ $dumpfile/$dumpvars สร้างไฟล์ VCD และรัน simulation ผ่าน ModelSim-Altera (vlib/vlog/vsim)
- ระบุและแก้ไขข้อผิดพลาดที่พบบ่อยในการเขียน testbench (race condition, blocking/non-blocking, X/Z propagation)
- อธิบายความแตกต่างระหว่าง RTL simulation, post-synthesis simulation และ post-implementation (timing) simulation

2. ความรู้พื้นฐานที่ต้องมีก่อนเข้าแล็บ

แล็บนี้ต่อยอดจากเนื้อหาบทที่ 8 (สไลด์บรรยาย) — ทบทวนโครงสร้าง testbench สี่ส่วนก่อนเริ่มลงมือทำ:



รูปที่ 1 — โครงสร้าง testbench สี่ส่วน: Clock/Reset Generator, Stimulus Generator, DUT, Checker/Scoreboard

2.1 หลักการ Self-Checking

testbench ที่ดีต้องตัดสิน PASS/FAIL ด้วยตัวเอง โดยเปรียบเทียบค่าที่ได้จริงจาก DUT กับค่าที่คำนวณไว้ล่วงหน้า แล้วรายงานผลด้วย $display — ไม่ใช่การนั่งไล่ดู waveform ทีละเส้นด้วยตา

2.2 System Tasks ที่ใช้บ่อยในแล็บนี้

System Task	หน้าที่
$display(...)	พิมพ์ข้อความทันทีที่ถูกเรียก — ใช้รายงาน PASS/FAIL
$dumpfile / $dumpvars	สร้างไฟล์ VCD สำหรับเปิดดู waveform ใน ModelSim
$time	คืนค่าเวลาปัจจุบันของ simulation
$finish	จบ simulation — testbench ทุกตัวต้องเรียกท้ายสุด

2.3 เครื่องมือที่ใช้ในแล็บนี้

- Quartus II — ใช้เฉพาะเปิดดูซอร์สโค้ด .v ของ DUT (ไม่ต้อง compile/program ลง DE0 สัปดาห์นี้)
- ModelSim-Altera — vlib, vlog, vsim, add wave, run -all (การจำลองเท่านั้น)

3. แบบฝึกหัด Debug และทบทวนความเข้าใจ

แบบฝึกหัดนี้ฝึกทักษะการ "อ่านและตรวจสอบ" โค้ดของคนอื่น ซึ่งสำคัญไม่แพ้การเขียนเอง — แต่ละ case study ด้านล่างให้ทั้งไฟล์ DUT (.v) ที่ถูกต้อง และไฟล์ testbench ที่มีบั๊กซ่อนอยู่ ให้หาบั๊กแต่ละจุด อธิบายอาการ แล้วเขียนวิธีแก้ในตารางถัดจากโค้ด:

3.1 แบบฝึกหัด Debug — ตรวจบัคหาให้เจอ

Case Study 1 — Shift Register (SIPO) (มีบั๊ก 3 จุด)

DUT: 4-bit Serial-In Parallel-Out Shift Register — ทุก posedge clk จะเลื่อนบิต sin เข้ามาทางขวา (LSB) แล้วเลื่อนบิตเดิมไปทางซ้าย:

VERILOG — shift_reg_sipo.v (DUT ให้มาแล้ว ถูกต้อง)

```
module shift_reg_sipo(
  input        clk, rst, sin,
  output reg [3:0] pout
);
  always @(posedge clk) begin
    if (rst)
      pout <= 4'b0000;
    else
      pout <= {pout[2:0], sin};
  end
endmodule
```

Testbench ด้านล่างมีบั๊กซ่อนอยู่ 3 จุด:

VERILOG — tb_shift_reg_buggy.v ที่มีบั๊ก

```
 1  module tb_shift_reg_buggy;
 2    reg clk=0, rst, sin;
 3    wire [3:0] pout;
 4    integer errors = 0;
 5
 6    shift_reg_sipo DUT(.clk(clk), .rst(rst), .sin(sin), .pout(pout));
 7    always #5 clk = ~clk;
 8
 9    task check_pout(input [3:0] exp, input [127:0] label);
10      begin
11        if (pout = exp) begin
12          $display("[PASS] %s: pout=%b", label, pout);
13        end else begin
14          $display("[FAIL] %s: expected=%b got=%b", label, exp, pout);
15          errors = errors + 1;
16        end
17      end
18    endtask
19
20    initial begin
21      rst=1; sin=0; @(posedge clk); rst=0;
22      check_pout(4'b0000, "after reset");
23      sin=1; @(posedge clk);
24      check_pout(4'b0001, "shift-in 1 (bit0)");
25      sin=0; @(posedge clk); #1;
26      check_pout(4'b0010, "shift-in 0 (bit0<-0)");
27      if (errors==0) $display("TESTBENCH PASSED");
28      else $display("FAILED: %0d errors", errors);
29    end
30  endmodule
```

บั๊กที่	บรรทัด	อาการ / สาเหตุ	วิธีแก้
1			
2			
3			

Case Study 2 — Priority Encoder (มีบั๊ก 2 จุด)

DUT: 4-to-2 Priority Encoder — บิตที่มีลำดับความสำคัญสูงสุด (ใกล้ MSB ที่สุด) ที่เป็น 1 จะกำหนดค่า out และ valid=1 เมื่อมี input เป็น 1 อย่างน้อยหนึ่งบิต:

VERILOG — priority_encoder.v (DUT ให้มาแล้ว ถูกต้อง)

```
module priority_encoder(
  input      [3:0] in,
  output reg [1:0] out,
  output reg       valid
);
  always @(*) begin
    valid = |in;
    casez (in)
      4'b1???: out = 2'd3;
      4'b01??: out = 2'd2;
      4'b001?: out = 2'd1;
      4'b0001: out = 2'd0;
      default: out = 2'd0;
    endcase
  end
endmodule
```

Testbench ด้านล่างมีบั๊กซ่อนอยู่ 2 จุด:

VERILOG — tb_priority_encoder_buggy.v ที่มีบั๊ก

```
 1  module tb_priority_encoder_buggy;
 2    reg  [3:0] in;
 3    wire [1:0] out;
 4    wire       valid;
 5    integer errors = 0;
 6
 7    priority_encoder DUT(.in(in), .out(out), .valid(valid));
 8
 9    task check_out(input [1:0] exp_out, input exp_valid, input [127:0] label);
10      begin
11        if (out = exp_out && valid == exp_valid) begin
12          $display("[FAIL] %s: expected out=%0d valid=%b got out=%0d
valid=%b",
13                   label, exp_out, exp_valid, out, valid);
14          errors = errors + 1;
15        end else begin
16          $display("[PASS] %s: out=%0d valid=%b", label, out, valid);
17        end
18      end
19    endtask
20
21    initial begin
22      in = 4'b0000; #10; check_out(2'd0, 1'b0, "no input active");
23      in = 4'b0001; #10; check_out(2'd0, 1'b1, "only bit0");
24      in = 4'b0110; #10; check_out(2'd2, 1'b1, "bit1,bit2 -> priority bit2");
25      in = 4'b1010; #10; check_out(2'd3, 1'b1, "bit3 highest priority");
```

```
26      if (errors==0) $display("TESTBENCH PASSED");
27      else $display("FAILED: %0d errors", errors);
28      $finish;
29    end
30  endmodule
```

บั๊กที่	บรรทัด	อาการ / สาเหตุ	วิธีแก้
1			
2			

3.2 แบบฝึกหัดถูก-ผิด — ทบทวนความเข้าใจ

พิจารณาข้อความต่อไปนี้ว่า ถูก หรือ ผิด แล้วเขียนคำตอบลงในช่องขวาสุด (ถ้าตอบว่าผิด ลองเขียนเหตุผลสั้นๆ กำกับไว้ข้างข้อความด้วย):

ข้อ	ข้อความ	ถูก / ผิด
1	Self-checking testbench ต้องอาศัยคนนั่งดู waveform เปรียบเทียบค่าด้วยตาเสมอ จึงจะตัดสิน PASS/FAIL ได้	
2	ควรใช้ === และ !== แทน == และ != ในการเปรียบเทียบค่าใน testbench เพื่อรองรับค่า X/Z ได้ถูกต้อง	
3	$dumpfile และ $dumpvars ใช้สำหรับสร้างไฟล์ .vcd เพื่อเปิดดู waveform ใน ModelSim	
4	ไฟล์ testbench (tb_*.v) จะถูก synthesize ลงบอร์ด DE0 เช่นเดียวกับ DUT	
5	การลบ #1 ที่อยู่หลัง @(posedge clk) ก่อนเรียก task ตรวจสอบผล จะไม่มีผลใดๆ ต่อผลการทดสอบ	
6	$finish ต้องถูกเรียกเพื่อจบการจำลอง มิฉะนั้น simulation อาจรันค้างไม่จบ	
7	Post-Implementation (Timing) Simulation ใช้ delay model ที่ใกล้เคียงวงจรจริงที่สุดในบรรดาการซิมทั้งสามระดับ	
8	ถ้า testbench ผ่าน RTL simulation แล้ว รับประกันได้ว่าวงจรจะทำงานถูกต้องบนบอร์ดจริงเสมอ	
9	State coverage หมายถึงการตรวจสอบว่า testbench ได้ทดสอบครบทุก "การเปลี่ยนสถานะ (transition)" ของ FSM แล้ว	
10	ลำดับคำสั่งที่ถูกต้องในการรัน ModelSim-Altera คือ vlib → vlog → vsim → run -all	

4. แบบฝึกหัดอุ่นเครื่อง — เขียน Testbench เอง (3 วงจร)

ก่อนเข้าสู่ self-checking testbench ในส่วนถัดไป ให้ฝึกเขียน testbench แบบพื้นฐานก่อนกับวงจร combinational ล้วน (ไม่มี clock) 3 วงจรด้านล่าง — แต่ละวงจรให้เขียน testbench เองทั้งหมดตั้งแต่ต้น (ไม่มีโค้ดให้เติม) แล้วป้อน input ให้ครบทุกชุดที่กำหนด บันทึกผลลัพธ์จริงจากการจำลองลงในตาราง จะได้โฟกัสที่กลไกการเขียน testbench เพียงอย่างเดียวก่อน (ยังไม่ต้องมี self-checking)

> เป้าหมายของแบบฝึกหัดนี้คือฝึก "เขียน" testbench ให้ครบองค์ประกอบและรันได้จริงกับวงจรที่มีลักษณะต่างกัน — การตัดสิน PASS/FAIL อัตโนมัติจะฝึกในส่วนที่ 1 และ 2 ถัดไป

4.1 วงจรที่ 1 — Full Adder

```
VERILOG — full_adder.v
module full_adder(
  input  a, b, cin,
  output sum, cout
);
  assign sum  = a ^ b ^ cin;
  assign cout = (a & b) | (b & cin) | (a & cin);
endmodule
```

งานที่ต้องทำ

เขียน testbench เองทั้งหมด ชื่อไฟล์ tb_full_adder.v โดยต้องมีองค์ประกอบครบดังนี้:

- module tb_full_adder; ... endmodule — ไม่มี port list
- ประกาศ reg สำหรับ a, b, cin และ wire สำหรับ sum, cout แล้ว instantiate DUT full_adder ด้วย named port mapping
- initial block ป้อนค่า input ให้ครบทุกชุดตามตารางด้านล่าง ทีละชุด โดยหน่วงเวลาด้วย #10 ระหว่างแต่ละชุด (ไม่ต้องมี clock generator เพราะเป็นวงจร combinational)
- ใช้ $monitor หรือ $display พิมพ์ค่า input/output ทุกครั้งที่ป้อนค่าใหม่ (ยังไม่ต้องเปรียบเทียบ/ตัดสิน PASS-FAIL ในขั้นนี้)
- เรียก $finish หลังป้อนครบทุกชุด

เขียนโค้ด Testbench ที่นี่

เขียนโค้ด tb_full_adder.v ฉบับเต็มด้วยลายมือลงในกล่องด้านล่าง (เขียนย่อ/กระชับได้ ไม่ต้องเว้นบรรทัดเป๊ะตามสไตล์ ถ้าพื้นที่ไม่พอให้เขียนต่อกระดาษแยกแล้วแนบเย็บติดกับหน้านี้):

[เขียนโค้ด tb_full_adder.v ที่นี่]

ตารางบันทึกผลลัพธ์ (Truth Table)

รันไฟล์ tb_full_adder.v ใน ModelSim แล้วบันทึกค่า output ที่ได้จริงจาก Transcript ลงในตาราง (input กำหนดไว้ให้ครบทุกชุดแล้ว):

#	a	b	cin	sum (จากการซิม)	cout (จากการซิม)
1	0	0	0		
2	0	0	1		
3	0	1	0		
4	0	1	1		
5	1	0	0		
6	1	0	1		
7	1	1	0		
8	1	1	1		

คำถามวิเคราะห์

ผลลัพธ์ sum และ cout ที่บันทึกได้ครบทั้ง 8 กรณี ตรงกับสมการ sum = a⊕b⊕cin และ cout = (a·b) + (b·cin) + (a·cin) หรือไม่ ถ้ามีกรณีที่ไม่ตรง ให้อธิบายว่าปัญหาน่าจะเกิดจากอะไร (โค้ด DUT ผิด หรือ testbench ป้อนค่า/อ่านค่าไม่ถูกต้อง)

4.2 วงจรที่ 2 — 2-to-1 Multiplexer

VERILOG — mux2to1.v

```
module mux2to1(
  input  sel, i0, i1,
  output y
);
  assign y = sel ? i1 : i0;
endmodule
```

งานที่ต้องทำ

เขียน testbench เองทั้งหมด ชื่อไฟล์ tb_mux2to1.v โดยต้องมีองค์ประกอบครบดังนี้:

- module tb_mux2to1; ... endmodule — ไม่มี port list
- ประกาศ reg สำหรับ sel, i0, i1 และ wire สำหรับ y แล้ว instantiate DUT mux2to1 ด้วย named port mapping
- initial block ป้อนค่า input ให้ครบทุกชุดตามตารางด้านล่าง ทีละชุด โดยหน่วงเวลาด้วย #10 ระหว่างแต่ละชุด (ไม่ต้องมี clock generator เพราะเป็นวงจร combinational)
- ใช้ $monitor หรือ $display พิมพ์ค่า input/output ทุกครั้งที่ป้อนค่าใหม่ (ยังไม่ต้องเปรียบเทียบ/ตัดสิน PASS-FAIL ในขั้นนี้)
- เรียก $finish หลังป้อนครบทุกชุด

เขียนโค้ด **Testbench** ที่นี่

เขียนโค้ด tb_mux2to1.v ฉบับเต็มด้วยลายมือลงในกล่องด้านล่าง (เขียนย่อ/กระชับได้ ไม่ต้องเว้นบรรทัดเป๊ะตามสไตล์ถ้าพื้นที่ไม่พอให้เขียนต่อกระดาษแยกแล้วแนบเย็บติดกับหน้านี้):

[เขียนโค้ด tb_mux2to1.v ที่นี่]

ตารางบันทึกผลลัพธ์ **(Truth Table)**

รันไฟล์ tb_mux2to1.v ใน ModelSim แล้วบันทึกค่า output ที่ได้จริงจาก Transcript ลงในตาราง (input กำหนดไว้ให้ครบทุกชุดแล้ว):

#	sel	i0	i1	y (จากการซิม)
1	0	0	0	
2	0	0	1	
3	0	1	0	
4	0	1	1	
5	1	0	0	
6	1	0	1	

#	sel	i0	i1	y (จากการชิม)
7	1	1	0	
8	1	1	1	

คำถามวิเคราะห์

ผลลัพธ์ y ที่บันทึกได้ครบทั้ง 8 กรณี ตรงกับพฤติกรรม "เมื่อ sel=0 ให้ y=i0 เสมอ, เมื่อ sel=1 ให้ y=i1 เสมอ" (ไม่ขึ้นกับอีกอินพุตที่ไม่ถูกเลือก) หรือไม่

4.3 วงจรที่ 3 — 2-to-4 Decoder with Enable

VERILOG — decoder2to4.v

```
module decoder2to4(
  input        en,
  input  [1:0] sel,
  output reg [3:0] y
);
  always @(*) begin
    if (en) begin
      case (sel)
        2'b00: y = 4'b0001;
        2'b01: y = 4'b0010;
        2'b10: y = 4'b0100;
        2'b11: y = 4'b1000;
      endcase
    end else y = 4'b0000;
  end
endmodule
```

งานที่ต้องทำ

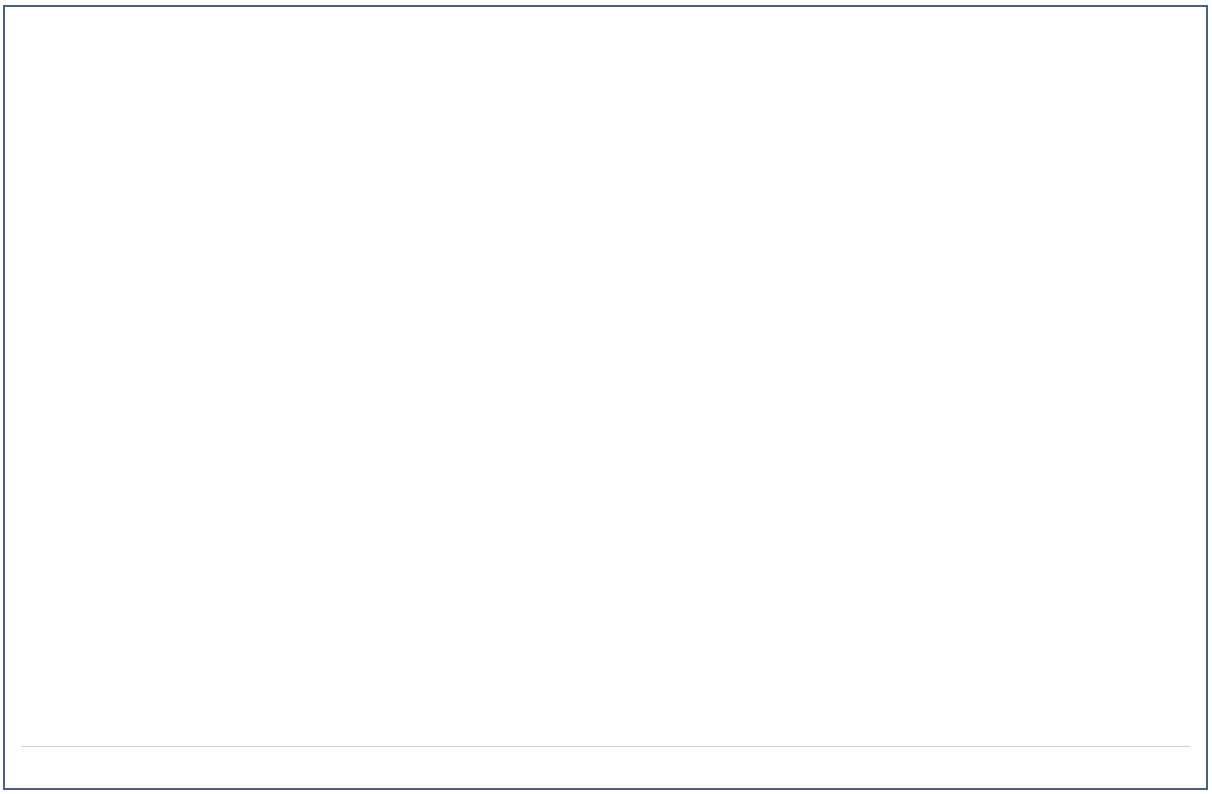
เขียน testbench เองทั้งหมด ชื่อไฟล์ tb_decoder2to4.v โดยต้องมีองค์ประกอบครบดังนี้:

- module tb_decoder2to4; ... endmodule — ไม่มี port list
- ประกาศ reg สำหรับ en, sel[1:0] และ wire [3:0] สำหรับ y แล้ว instantiate DUT decoder2to4 ด้วย named port mapping
- initial block ป้อนค่า input ให้ครบทุกชุดตามตารางด้านล่าง ทีละชุด โดยหน่วงเวลาด้วย #10 ระหว่างแต่ละชุด (ไม่ต้องมี clock generator เพราะเป็นวงจร combinational) — วงจรนี้ใช้ always @(*) กับ case แทน assign ตรงๆ เหมือนสองวงจรก่อนหน้า แต่วิธีเขียน testbench ยังเหมือนเดิม
- ใช้ $monitor หรือ $display พิมพ์ค่า input/output ทุกครั้งที่ป้อนค่าใหม่ (ยังไม่ต้องเปรียบเทียบ/ตัดสิน PASS-FAIL ในขั้นนี้)
- เรียก $finish หลังป้อนครบทุกชุด

เขียนโค้ด Testbench ที่นี่

เขียนโค้ด tb_decoder2to4.v ฉบับเต็มด้วยลายมือลงในกล่องด้านล่าง (เขียนย่อ/กระชับได้ ไม่ต้องเว้นบรรทัดเป๊ะตามสไตล์ถ้าพื้นที่ไม่พอให้เขียนต่อกระดาษแยกแล้วแนบเย็บติดกับหน้านี้):

[เขียนโค้ด tb_decoder2to4.v ที่นี่]



ตารางบันทึกผลลัพธ์ (Truth Table)

รันไฟล์ tb_decoder2to4.v ใน ModelSim แล้วบันทึกค่า output ที่ได้จริงจาก Transcript ลงในตาราง (input กำหนดไว้ให้ครบทุกชุดแล้ว):

#	en	sel[1:0]	y[3:0] (จากการซิม)
1	0	2'b00	
2	0	2'b01	
3	0	2'b10	
4	0	2'b11	
5	1	2'b00	
6	1	2'b01	
7	1	2'b10	
8	1	2'b11	

คำถามวิเคราะห์

เมื่อ en=0 ค่า y ที่ได้จากการซิมเป็น 4'b0000 ทุกกรณีของ sel หรือไม่ และเมื่อ en=1 ค่า y ที่ได้ตรงกับรูปแบบ one-hot ตามค่า sel หรือไม่ (มี bit ใดเป็น 1 มากกว่าหนึ่งบิตพร้อมกันหรือไม่)

5. ส่วนที่ 1 — Self-Checking Testbench: Mod-10 Up/Down Counter

DUT ที่ใช้คือ Mod-10 Up/Down Counter จากสัปดาห์ที่ 6 (updown_counter.v) — port: clk, rst, en, updown (input), count[3:0] (output)

VERILOG — updown_counter.v (DUT ให้มาแล้ว)

```
module updown_counter(
  input        clk, rst, en, updown,
  output reg [3:0] count
);
  always @(posedge clk) begin
    if (rst)
      count <= 4'd0;
    else if (en) begin
      if (updown) begin
        if (count == 4'd9) count <= 4'd0;
        else               count <= count + 4'd1;
      end else begin
        if (count == 4'd0) count <= 4'd9;
        else               count <= count - 4'd1;
      end
    end
  end
endmodule
```

5.1 แผนทดสอบ (Test Plan)

ระบุกรณีทดสอบและการกระตุ้นที่ต้องป้อนให้ครบ — ค่าที่คาดว่าจะได้ ให้คิดเองจากพฤติกรรมของ DUT ด้านบน แล้วเขียนเป็นเงื่อนไขตรวจสอบใน testbench เอง (ไม่ระบุคำตอบไว้ในตารางนี้):

กรณีทดสอบ	การกระตุ้น (stimulus)
Reset	rst = 1 หนึ่ง clock edge
นับขึ้นปกติ	en=1, updown=1 ให้ครบ 10 clock
นับลงปกติ	en=1, updown=0 หนึ่ง clock จาก 0
Enable = 0	en = 0 ระหว่างนับ

5.2 งานที่ต้องทำ — เติมโค้ดในช่องว่าง

เติมโค้ดในช่องว่าง (1)–(6) ให้ testbench ทำงานถูกต้องตามแผนทดสอบข้อ 5.1 แล้วบันทึกเป็นไฟล์ tb_updown_counter.v

VERILOG — tb_updown_counter.v (เติมคำในช่องว่าง)

```
`timescale 1ns/1ps
module tb_updown_counter;
  reg clk = 0, rst, en, updown;
  wire [3:0] count;
  integer errors = 0;

  updown_counter DUT (
    .clk(clk), .rst(rst), .en(en), .updown(updown), .count(count)
  );

  always #5 clk = _______________;                    // (1)
```

```
  task check_count(input [3:0] exp, input [127:0] label);
    begin
      if (count ______ exp) begin                        // (2)
        $display("[FAIL] %s: expected=%0d got=%0d @%0t", label, exp, count,
$time);
        errors = errors + 1;
      end else
        $display("[PASS] %s: count=%0d", label, count);
    end
  endtask

  initial begin
    $dumpfile("tb_counter.vcd"); $dumpvars(0, tb_updown_counter);

    rst=1; en=0; updown=1; @(posedge clk); ______;      // (3)
    check_count(4'd0, "after reset");

    rst=0; en=1; __________ (10) @(posedge clk); #1;    // (4)
    check_count(4'd0, "wrap 9->0 after 10 counts");

    updown=0; @(posedge clk); #1;
    check_count(______, "wrap 0->9 counting down");    // (5)

    if (errors==0) $display("TESTBENCH PASSED");
    else $display("FAILED: %0d errors", errors);
    ______;                                             // (6)
  end
endmodule
```

ตารางคำตอบช่องว่าง:

ช่องว่าง	คำตอบของนักศึกษา
(1)	
(2)	
(3)	
(4)	
(5)	
(6)	

5.3 ขั้นตอนการรันใน ModelSim-Altera

- 1. vlib work
- 2. vlog updown_counter.v tb_updown_counter.v
- 3. vsim tb_updown_counter
- 4. add wave -radix binary /tb_updown_counter/*
- 5. run -all

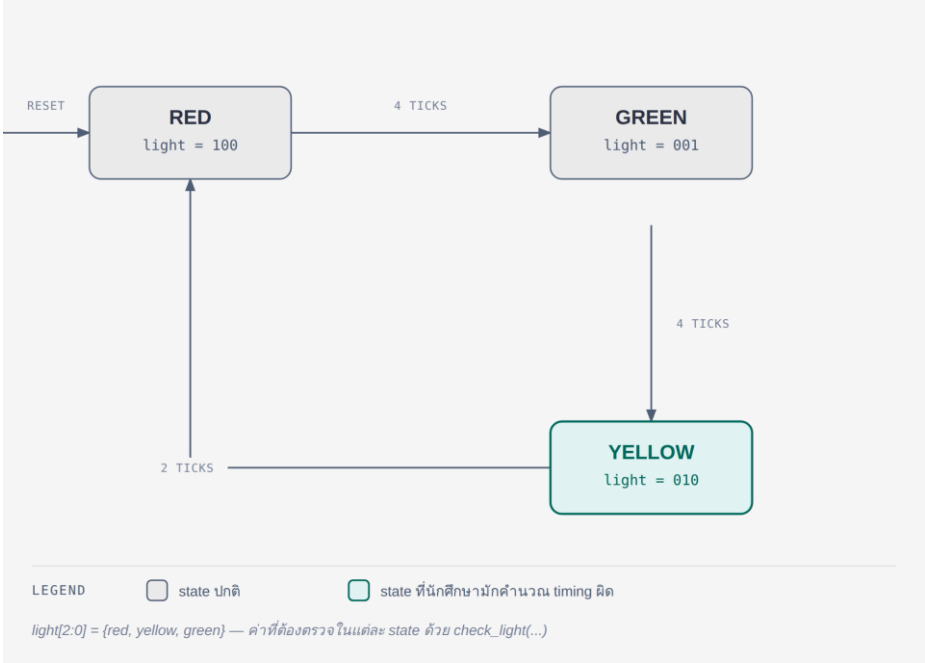
5.4 บันทึกผลลัพธ์

ดู Transcript window ใน ModelSim แล้วคัดลอกข้อความด้วยลายมือลงในกล่องด้านล่าง ให้ครบทุกบรรทัด [PASS]/[FAIL] และบรรทัด "TESTBENCH PASSED" ที่ท้ายสุด:

[คัดลอกข้อความจาก Transcript ที่นี่]

6. ส่วนที่ 2 — Self-Checking Testbench: Traffic Light FSM (Moore)

DUT ที่ใช้คือ Traffic Light Controller (Moore FSM) จากสัปดาห์ที่ 7 — port: clk, rst, tick (input), light[2:0] (output, {red,yellow,green})



รูปที่ 2 — State diagram ของ Traffic Light Controller และค่า light[2:0] ที่ต้องตรวจในแต่ละ state

VERILOG — traffic_light.v (DUT ให้มาแล้ว)

```
module traffic_light_fsm(
  input        clk, rst, tick,
  output [2:0] light
);
  localparam RED=2'd0, GREEN=2'd1, YELLOW=2'd2;
  reg [1:0] state;
  reg [2:0] cnt;

  always @(posedge clk) begin
    if (rst) begin
      state <= RED; cnt <= 3'd0;
    end else if (tick) begin
      case (state)
        RED:    if (cnt==3'd3) begin state<=GREEN;  cnt<=0; end else cnt<=cnt+1;
        GREEN:  if (cnt==3'd3) begin state<=YELLOW; cnt<=0; end else cnt<=cnt+1;
        YELLOW: if (cnt==3'd1) begin state<=RED;    cnt<=0; end else cnt<=cnt+1;
        default: state <= RED;
      endcase
    end
  end

  assign light = (state==RED)    ? 3'b100 :
                 (state==GREEN)  ? 3'b001 :
                 (state==YELLOW)? 3'b010 : 3'b100;
endmodule
```

6.1 แผนทดสอบ (Test Plan)

อ้างอิงพฤติกรรมจากรูปที่ 2 (state diagram) ด้านบน — ระบุกรณีทดสอบและการกระตุ้นให้ครบ (ค่า light[2:0] และจำนวน tick ที่ต้องอยู่ในแต่ละ state ให้ดูจากรูปที่ 2 เอง ไม่ระบุคำตอบซ้ำไว้ในตารางนี้):

กรณีทดสอบ	การกระตุ้น (stimulus)
หลัง reset	rst = 1 หนึ่ง clock edge
RED → GREEN	tick ให้ครบจำนวนที่ต้องอยู่ใน state RED (ดูรูปที่ 2)
GREEN → YELLOW	tick ให้ครบจำนวนที่ต้องอยู่ใน state GREEN (ดูรูปที่ 2)
YELLOW → RED (วนกลับ)	tick ให้ครบจำนวนที่ต้องอยู่ใน state YELLOW (ดูรูปที่ 2)

6.2 งานที่ต้องทำ — เติมโค้ดในช่องว่าง

เติมโค้ดในช่องว่าง (1)–(6) ให้ตรงกับพฤติกรรมในรูปที่ 2 และแผนทดสอบข้อ 6.1 แล้วบันทึกเป็นไฟล์ tb_traffic_light.v

VERILOG — tb_traffic_light.v (เติมคำในช่องว่าง)

```
`timescale 1ns/1ps
module tb_traffic_light;
  reg clk=0, rst, tick;
  wire [2:0] light;
  integer errors=0;

  traffic_light_fsm DUT (.clk(clk), .rst(rst), .tick(tick), .light(light));
  always #5 clk = ~clk;

  task pulse_tick;
    begin tick=1; @(posedge clk); tick=______; end       // (1)
  endtask

  task check_light(input [2:0] exp, input [127:0] label);
    begin
      if (light !== exp) begin
        $display("[FAIL] %s: expected=%b got=%b @%0t", label, exp, light,
$time);
        errors = errors + 1;
      end else $display("[PASS] %s: light=%b", label, light);
    end
  endtask

  initial begin
    rst=1; tick=0; @(posedge clk); rst=0; #1;
    check_light(3'b___, "reset -> RED");                // (2)

    repeat (____) pulse_tick; #1;                        // (3)
    check_light(3'b001, "RED(4)->GREEN");

    repeat (4) pulse_tick; #1;
    check_light(3'b___, "GREEN(4)->YELLOW");             // (4)

    repeat (2) pulse_tick; #1;
    check_light(3'b100, "YELLOW(__)->RED (loop)");       // (5)

    if (errors==0) $display("TESTBENCH PASSED");
    ______;                                               // (6)
  end
endmodule
```

ตารางคำตอบช่องว่าง:

ช่องว่าง	คำตอบของนักศึกษา
(1)	
(2)	
(3)	
(4)	
(5)	
(6)	